COMP4442 Service and Cloud Computing

Tutorial 3

LI Yuhui

merrill.li@connect.polyu.hk

Jan 28, 2026

Outline

- What is Docker? Why Docker?
- How to containerize your application?
- Some knowledge about Docker' design

Outline

- What is Docker? Why Docker?
- How to containerize your application?
- Some knowledge about Docker' design

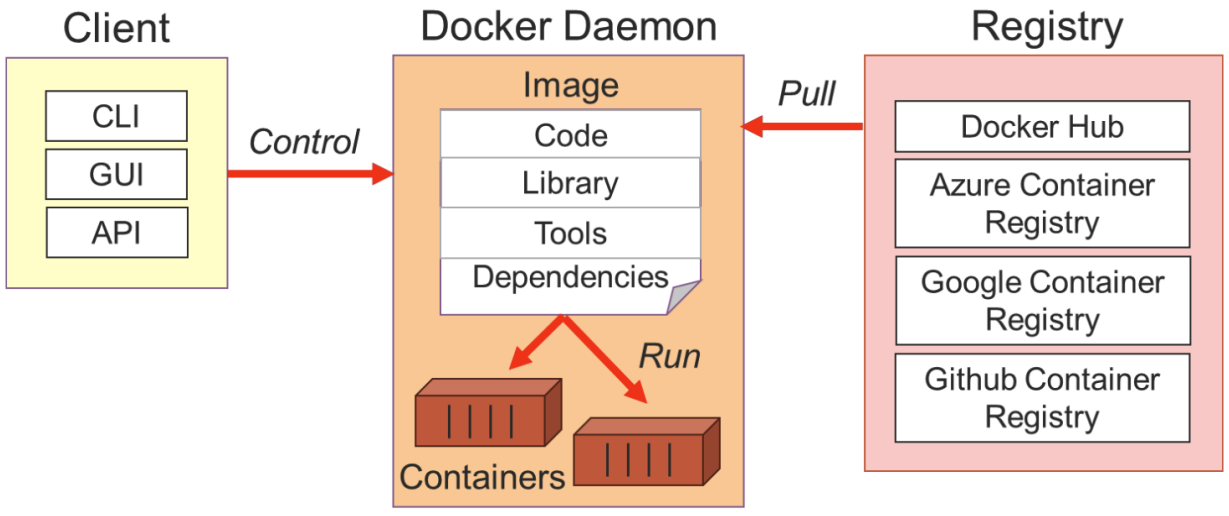
Question 1: What is Docker?

- An open-source software created by Solomon Hykes
  - ✓ Docker is a well known containerization software
  - ✓ Containerization is a technique that combines the application, related dependencies, and system libraries organized to build in the form of a container

- Using Docker, software developers can easily create applications, and wrap those applications in a special package called a container image
  - ✓ Available in different platforms (Windows, Mac and Linux)
  - ✓ Not CORE! (Compile Once Run Everywhere)

- Docker standardizes the process of running any service on any local development environment

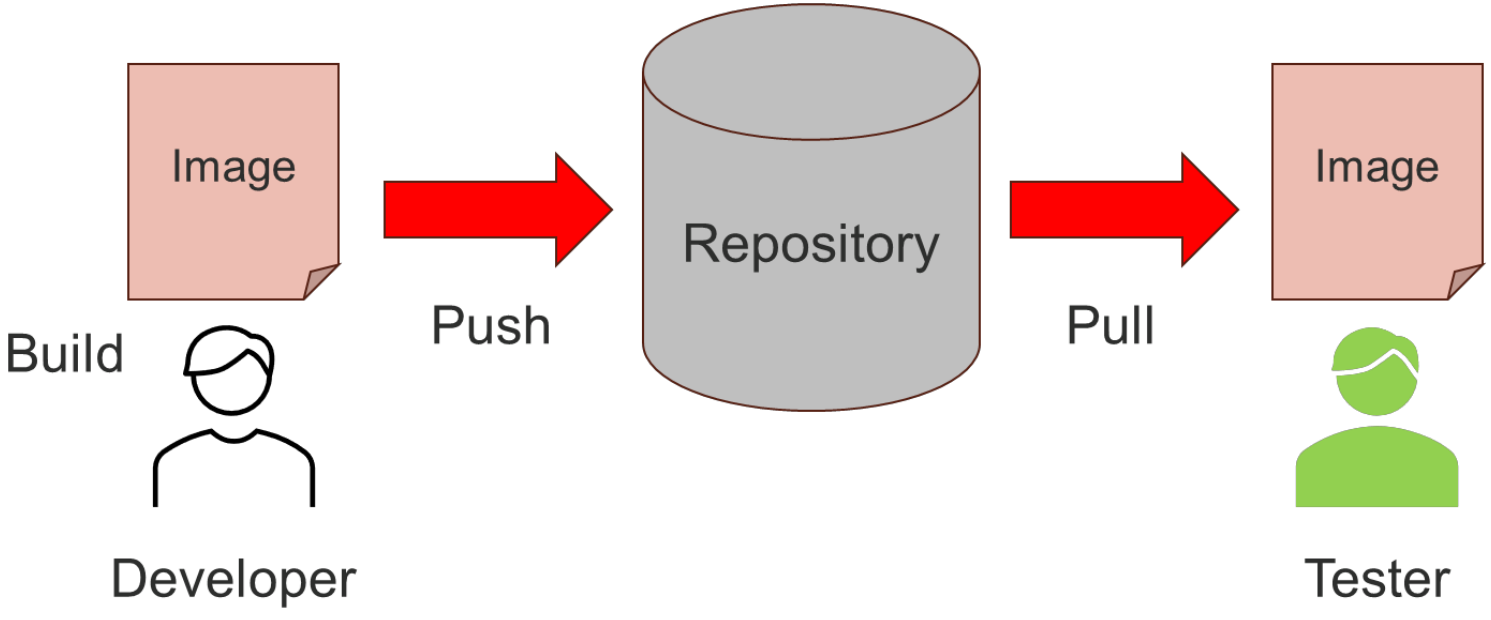
How does Docker work?



- Docker packages the application code, libraries and dependencies, configuration, system tools and runtime necessary for an application to run into an image
- To run the containerized software, simply run the image on Docker to create a container
  - ✓ A container is a runnable instance of a container image. You can run multiple containers of an image, or different versions of the same application at the same time (i.e. from different images)
- Image is
  - ✓ Immutable – it contains layers of instructions that defines the blueprint of a container.When a container is run from an image, Docker adds a writable layer on top of the image layers where all changes are made.
  - ✓ Can be distributed – Images may be stored in a public registry; this facilitates sharing and distribution

Containerization

- Instead of downloading applications from others, you can also make your own application run in Docker
  - This is known as dockerization or containerization, which means to package your application and the necessary configurations, dependencies, runtime, etc. into an image
  - Dockerization is common in real life, especially when you want to let your teammate do the testing in the same environment



Outline

- What is Docker? Why Docker?
- How to containerize your application?
- Some knowledge about Docker' design

How to containerize an Application?

- Containerization requires one or more Dockerfiles, each of which is actually the packaging text (instructions) that tells Docker what to do when the image is built
  - Docker specifies the syntax of Dockerfile
  - https://docs.docker.com/reference/dockerfile/

- When an application needs multiple containers (e.g. a web application that works with a database), a Docker Compose file is usually included that contains the configuration settings for all the containers of the application
  - https://docs.docker.com/compose/compose-file/

Dockerfile: An Example

```
FROM python:3.9-slim-buster

WORKDIR /app

# 复制应用程序代码和类别标签文件
COPY app.py .
COPY imagenet_classes.txt .
COPY resnet34-weights.pth .

# 安装 PyTorch 和 torchvision
RUN pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu

# 安装 Flask 和 Pillow
RUN pip install Flask Pillow

# 暴露 Web 服务端口
EXPOSE 7099

# 定义容器启动命令
CMD ["python", "app.py"]
```

Dockerfile: Syntax of FROM

```
FROM python:3.9-slim-buster

WORKDIR /app

# 复制应用程序代码和类别标签文件
COPY app.py .
COPY imagenet_classes.txt .
COPY resnet34-weights.pth .

# 安装 PyTorch 和 torchvision
RUN pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

- Syntax: **FROM <image_name>[:<tag>]**
  - The image_name must be provided, and it is usually a base image from a public repository
  - The base image will be pulled automatically when the image of your application is built
  - The tag is optional, depending on whether the base image has specified it
    - If not provided, assume the 'latest' tag by default
- After the FROM line, the other lines are run sequentially like a shell script

Dockerfile: Syntax of WORKDIR

```
FROM python:3.9-slim-buster

WORKDIR /app

# 复制应用程序代码和类别标签文件
COPY app.py .
COPY imagenet_classes.txt .
COPY resnet34-weights.pth .

# 安装 PyTorch 和 torchvision
RUN pip install torch torchvision torchaudio --
index-url https://download.pytorch.org/whl/cpu
```

- WORKDIR changes the working directory for the later instructions to work on.
  - ✓ Instructions such as RUN, CMD, ENTRYPOINT, COPY and ADD
- The target directory is specified as the argument
  - ✓ So it is similar to the 'cd' command in Linux or Windows command prompt
  - ✓ If the argument starts with /, the path is an absolute path, otherwise, the path is considered relative (starting from the current directory)
  - ✓ If you want to go to the parent directory, use 'WORKDIR ..'
- If the target WORKDIR doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.

Dockerfile: Syntax of COPY

```
FROM python:3.9-slim-buster

WORKDIR /app

# 复制应用程序代码和类别标签文件
COPY app.py .
COPY imagenet_classes.txt .
COPY resnet34-weights.pth .

# 安装 PyTorch 和 torchvision
RUN pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

- The COPY instruction copies file(s) from your host machine (argument 1) to the container image (argument 2)
- If argument 1 is a filename without any path, the file must exist in your working directory of the host
  - If argument 1 is with a relative path, it should be relative to your working directory, but ancestor directory paths are not allowed (i.e., ./somedir/afile, ../otherdir/bfile)
- argument 1 is a directory
  - argument 2 is './', the contents of the source directory will be copied to working directory of the container; no directory will be created
  - argument 2 is name, a new directory with be created if it does not exist, and the contents of the source directory will be copied to there
- Wildcards (? and *) are allowed for argument 1

Dockerfile: Syntax of RUN

```
FROM python:3.9-slim-buster

WORKDIR /app

# 复制应用程序代码和类别标签文件
COPY app.py .
COPY imagenet_classes.txt .
COPY resnet34-weights.pth .

# 安装 PyTorch 和 torchvision
RUN pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

- RUN executes a command in the container
- RUN can be followed by a command in Linux command format, this is known as the shell form
  - /bin/sh is the default shell, to change it, specify the shell path in between RUN and the command
- Example: creates a folder app under the working directory
  - **RUN mkdir app**
- RUN can also take the exec form:
  - **RUN ["prog", "param1", "param2", …, "param-n"]**

- **RUN is "stateless"!!! Subsequent commands are not executed in one session!**

Dockerfile: Syntax of EXPOSE

```
# 安装 Flask 和 Pillow
RUN pip install Flask Pillow

# 暴露 Web 服务端口
EXPOSE 7099

# 定义容器启动命令
CMD ["python", "app.py"]
# ENTRYPOINT ["python", "app.py"]
```

- The EXPOSE instruction informs Docker that the container listens to the specified port number
  - This also lets the user know which port on the container is to be published when the 'docker run' command is executed
- Syntax: EXPOSE <port_no>[/<protocol>]
  - /protocol is optional, if not specified, TCP is used
  - Other examples:

```
# Expose Port 8282 on TCP
EXPOSE 8282
# Expose Port 80 on UDP
EXPOSE 80/udp
```

Dockerfile: Syntax of ENTRYPOINT

```
# 安装 Flask 和 Pillow
RUN pip install Flask Pillow

# 暴露 Web 服务端口
EXPOSE 7099

# 定义容器启动命令
CMD ["python", "app.py"]
# ENTRYPOINT ["python", "app.py"]
```

- Like RUN, ENTRYPOINT also executes a command
  - The syntax for both RUN and ENTRYPOINT are the same, and they both support shell form and exec form
- But RUN executes the command when the user executes 'docker build', while ENTRYPOINT executes the command when the user executes 'docker run'
  - ENTRYPOINT is used to execute the application in the container, while RUN is used to execute the commands related to the image building process
- Therefore, ENTRYPOINT allows us to configure a container that will run as your target application

Dockerfile: Syntax of CMD

- CMD can be used for setting the command to be executed when running a container from an image, hence <u>many people use it instead of ENTRYPOINT</u>
  - Again, they have the same syntax, and they both support the shell form and exec form
- CMD can be used as a way to provide default arguments to the ENTRYPOINT instruction
  - If the user specifies arguments to “docker run” then they will override the default specified in CMD, but still use the default ENTRYPOINT
  - If CMD is used to provide default arguments for the ENTRYPOINT instruction, both the CMD and ENTRYPOINT instructions should be specified in the exec form

```
CMD ["executable","param1","param2"] (exec form)
CMD ["param1","param2"] (exec form, as default parameters to ENTRYPOINT)
CMD command param1 param2 (shell form)
```

Dockerfile: ENTRYPOINT vs CMD

- Both CMD and ENTRYPOINT instructions define what command gets executed when running a container.
- There are few rules that describe their co-operation
  - Every Dockerfile can only has at most one ENTRYPOINT and one CMD instruction
    - If there are >1 ENTRYPOINT or >1 CMD instructions, only the last one will be effective
  - ENTRYPOINT should be defined when using the container as an executable
  - CMD should be used as a way of defining default arguments for an ENTRYPOINT command or for executing an ad-hoc command in a container.
  - CMD will be overridden when running the container with alternative arguments

Dockerfile: Syntax of ENV

- The ENV instruction sets an environment variable in the container, e.g.:
  - **ENV MY_NAME="John Doe"**
- You may specify multiple environment variables in one ENV instruction, e.g.:
  - **ENV MY_NAME="John Doe" && MY_ID="345"**
- This is better than using multiple ENV instructions, as each ENV instruction creates a new shell layer

Build an Image using Dockerfile

- Use the "docker build" command in the Command Prompt or Terminal to build an image, an example:
  - You must specify at the end of the command the path where the Dockerfile is, in this example, the current directory:

```
docker build -f Dockerfile -t lc-resnet-service:v0.1 .
```

  - The path can also be a web URL (without http:// or https://) that contains a repository and the Dockerfile, in this case, the repository will be cloned and built in your Docker environment, example:

```
docker build github.com/creack/docker-firefox
```

Build an Image using Dockerfile

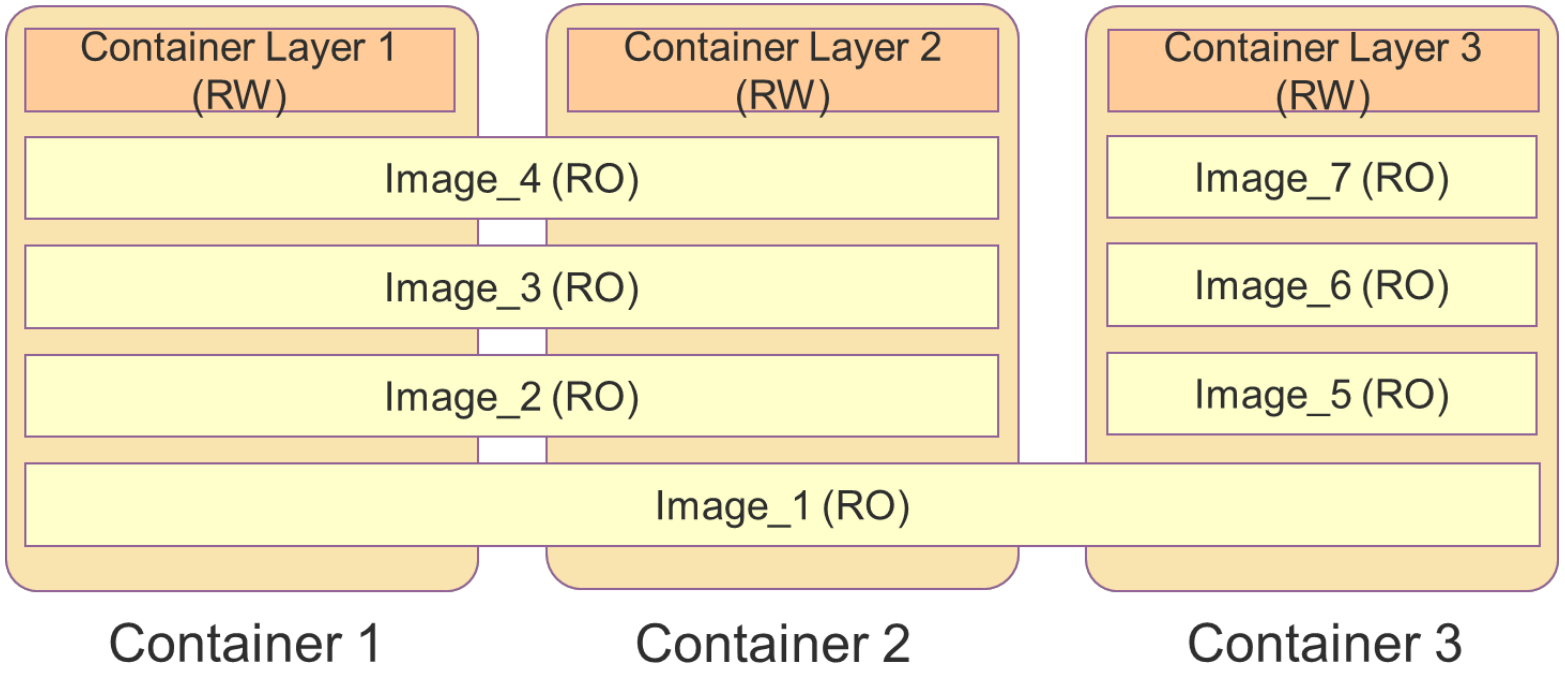
- Some frequently used options include:
  - -t <image_name>:<tag_name>
    - ➢ specifies the image and tag names
  - -f <dockerfile_name>
    - ➢ specifies the path name and file name of the Dockerfile
    - ➢ If not specified, **the default name Dockerfile will be assumed**
- See the following URL for more options:

  https://docs.docker.com/engine/reference/commandline/build/

Outline

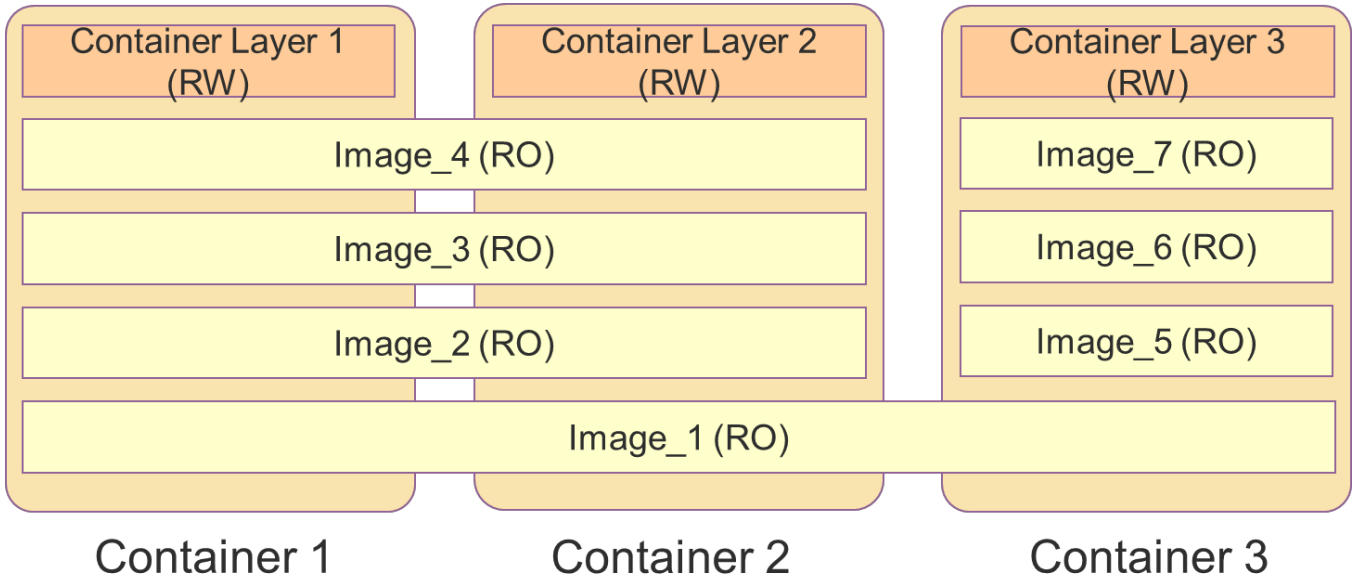
- What is Docker? Why Docker?
- How to containerize your application?
- Some knowledge about Docker' design

Layered File System of Containers



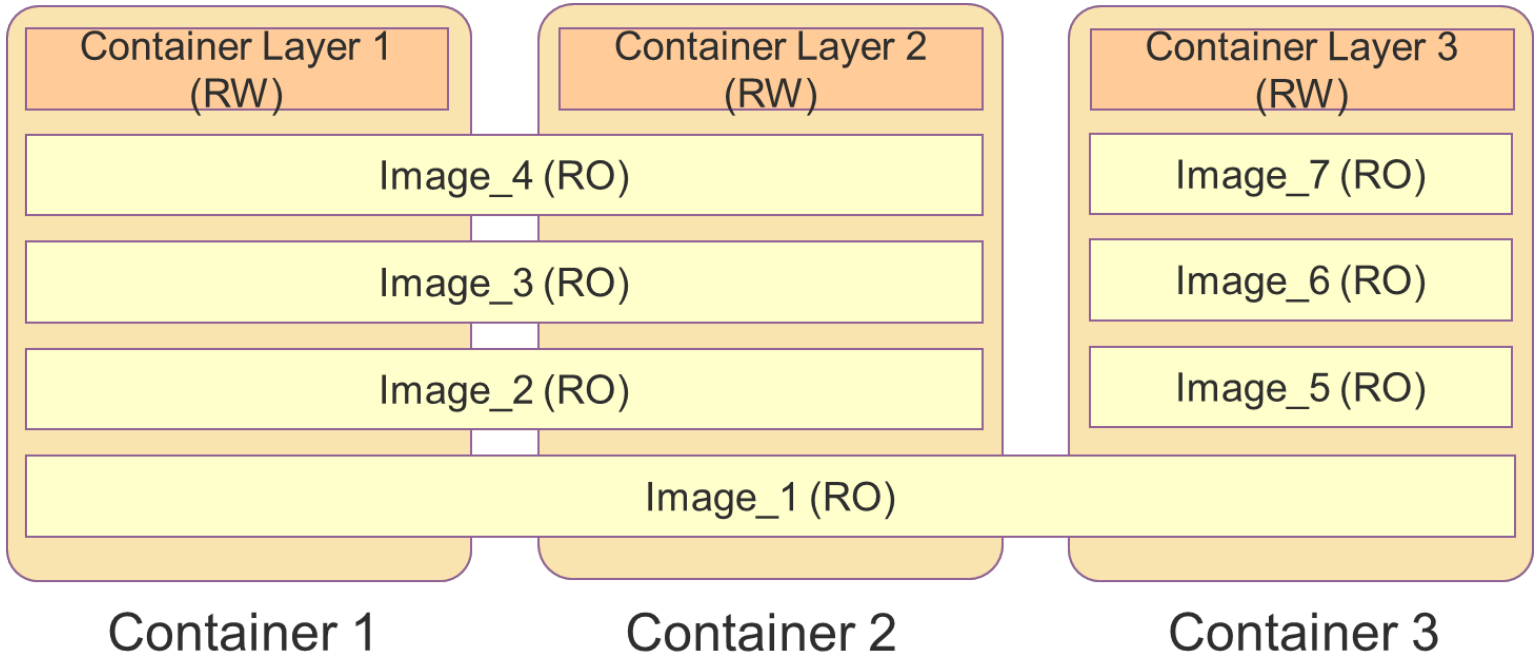
- The first line of a Dockerfile always starts with FROM
  - This specifies the base image that provides the operating system environment for running your application

Layered File System of Containers



- Docker images work in layers, it is common for an application to base on an image, which in turn bases on another layer image
  - Roughly speaking, each instruction in a Dockerfile translates to an image layer
  - The based images are read-only, and only one copy of the image is needed even if multiple containers use the same image
- Each container has a container image layer with read-write permission, for storing all the modifications to the filesystem.
  - Such modifications are highly not recommended to use as persistent storage!

Layered File System of Containers



- Image layers
  - Each layer in an image contains a set of filesystem changes - additions, deletions, or modifications
  - This is beneficial because it allows layers to be reused between images
- Cached layers
  - The concept of layers comes in handy at the time of building images. Because layers are intermediate images, if you make a change to your Dockerfile, docker will rebuild only the layer that was changed and the ones after.
  - Therefore, ordering the Dockerfile instructions appropriately helps us avoid unnecessary work at build time